

Prompting overview

Prompting is how you tell ChatGPT what you want to know, make, or change. A prompt can be a question, an instruction, or a goal. You don't need technical syntax or a rigid formula. Start in your own words, review the response, and use follow-up messages to shape the result.

A short prompt is often enough. For larger or more important tasks, include the parts that matter:

- **Goal:** What should ChatGPT do?
- **Context:** What information or sources will help?
- **Output:** What format, length, or level of detail do you need?
- **Boundaries:** What must stay unchanged? What should ChatGPT avoid or check with you before it acts?

Use only the parts that help. You don't need to fill in every item or follow a required format.

Describe the result you need

Start with the result, not a detailed list of steps. Include the audience or format when those details change what ChatGPT should produce.

Turn these meeting notes into a short update for the project team.
Put the decisions and next steps first.



This prompt explains what to create and who will read it. Describe a process when the process itself matters. Otherwise, leave ChatGPT room to search, compare information, and adjust its approach.

Add useful context

Share the information that could change the result. Add only the sources that matter, and explain what ChatGPT should take from each one.

Ask AI

Loading docs agent...

choose from the tools available to it. In ChatGPT, type `@` in the composer to choose a specific plugin.



Learn about plugins

Find, install, and use plugins in ChatGPT and Codex.

Personalize ChatGPT

Put preferences that should apply across chats in **Settings > Personalization** as custom instructions. Keep details that matter only to the current chat in the prompt.



Review personalization settings

Set a default personality, custom instructions, and other app preferences.

Loading docs agent...

For important work, ask ChatGPT for a final check, such as confirming every action item has an owner and due date or flagging information it couldn't verify. Then review the result yourself before you use or share it.

Improve the result with follow-up messages

Your first prompt doesn't need to be perfect. Review the result, then ask for the specific change you want.

Make the opening more direct, keep the evidence, and move the recommendation above the background section.



You can add a missing source, correct the direction, ask for another option, or change the level of detail without starting over.

Loading docs agent...

Lead with the decisions leadership needs to make and the next steps. Summarize progress, risks, owners, and due dates. Keep approved dates and budget figures unchanged. Flag any conflicting or missing information, and don't send or publish anything.

Before you finish, check that every next step has an owner and due date.

This prompt covers the **Goal, Context, Output**, and **Boundaries**, then asks for a final check without spelling out every step.

Use voice dictation

Loading docs agent...

Explain how compound interest works for someone who has never invested.
Use one concrete example and define any financial terms you introduce.



Draft and refine writing

Draft a friendly email declining this invitation because I will be traveling.
Keep it under 120 words and leave the door open for a future event.



Compare options

Loading docs agent...

improves quality, or helps you make an important decision.

Start with one result you can review:

- Include only relevant sources and limit the date range when appropriate.
- Define the audience, output format, and desired length.
- Separate required work from optional improvements or polish.
- Ask for a plan when the approach matters. Require your approval before ChatGPT sends, publishes, or changes information other people rely on.
- Narrow or stop the task if it starts doing work you no longer need.

Review the first result, refine the instructions, and reuse the workflow when it works.

Turn source material into finished files

Loading docs agent...

for each day. I'd tag any missing decisions before producing the final files.

For recurring work, first refine the prompt in a normal chat. After the output is reliable, [schedule a task inside that chat](#). Create a standalone scheduled task instead when each scheduled run should start a new chat.

Prompting Codex

Use Codex when you want ChatGPT to work with code, a codebase, or developer tools. A useful Codex prompt names the behavior you want, points to the relevant code or reproduction steps, preserves important constraints, and says how to verify the change.

For a multi-step task, enter `/plan` in the app composer when you want Codex to investigate and propose an approach before editing. When [Goal mode](#) is available,

Loading docs agent...

- 1 Open the most relevant files.
- 2 Select the code you care about (optional but recommended).
- 3 Prompt Codex:

Explain how the request flows through the selected code.



Include:

- a short summary of the responsibilities of each module involved
- what data is validated and where
- one or two "gotchas" to watch for when changing this

Loading docs agent...

Fix a bug

Use this when you have a failing behavior you can reproduce locally.

CLI workflow (tight loop with reproduction and verification)

- 1 Start Codex at the repo root:

```
codex
```



- 2 Give Codex a reproduction recipe, plus the file(s) you suspect:

Loading docs agent...

if you have a standard check pipeline, ask it to run it.

After the fix, run lint + the smallest relevant test suite.
Report the commands and results.



IDE extension workflow

- 1 Open the file where you think the bug lives, plus its nearest caller.
- 2 Prompt Codex:

Find the bug causing "Saved" to show without persisting changes. After proposing the fix, tell me how to verify it in the UI.



Loading docs agent...

codex



2 Prompt with a function name:

Add a test for the `invert_list` function in `@transform.ts`.
Cover the happy path plus edge cases.



Prototype from a screenshot

Use this when you want to turn a design mock, screenshot, or UI reference into a working prototype.

CLI workflow (image + prompt)

Loading docs agent...

implementation constraints (framework, routing, component style).

- Include behavior the image doesn't show in text, such as hover states, validation rules, or keyboard interactions.

Verification:

- Ask Codex to run the dev server (if allowed) and tell you exactly where to look:

Start the dev server and tell me the local URL/route to view the prototype.



IDE extension workflow (image + existing files)

Loading docs agent...

3 Prompt Codex to make changes:

Propose 2–3 styling improvements for the landing page.



4 Pick a direction and iterate with small, specific prompts:

Go with option 2.



Change only the header:

- make the typography more editorial
- increase whitespace
- ensure it still looks good on mobile

Loading docs agent...

\$plan



We need to refactor the auth subsystem to:

- split responsibilities (token parsing vs session loading vs permissions)
- reduce circular imports
- improve testability

Constraints:

- No user-visible behavior changes
- Keep public APIs stable
- Include a step-by-step migration plan

3 Review the plan and negotiate changes:

Loading docs agent...

- 5 Create a PR directly from the cloud or pull changes locally to test and finish up.
- 6 Iterate on additional milestones of the plan.

Tasks delegated to the cloud run in isolated environments. Internet access is off during the agent phase unless you enable it for the environment. Learn more about [cloud internet access](#).

Do a local code review

Use this when you want a second set of eyes before committing or creating a PR.

CLI workflow (review your working tree)

- 1 Start Codex:

Loading docs agent...

1 Open the pull request on GitHub.

2 Leave a comment that tags Codex with explicit focus areas:

@codex review



3 Optional: Provide more explicit instructions.

@codex review for security vulnerabilities and security concerns



Update documentation

Loading docs agent...